Agentic Systems Design and Development

CIDM 6333

Orientation · Fall 2026

SURVEY BUILD GOVERN DEFEND

Dr. Jeffry Babb · Paul and Virginia Engler College of Business · West Texas A&M University

The question behind the course

Why are agentic systems built this way, and can you answer for yours?

Survey

Each week we take two or three real systems apart against one component model. You write one Design Analysis per week.

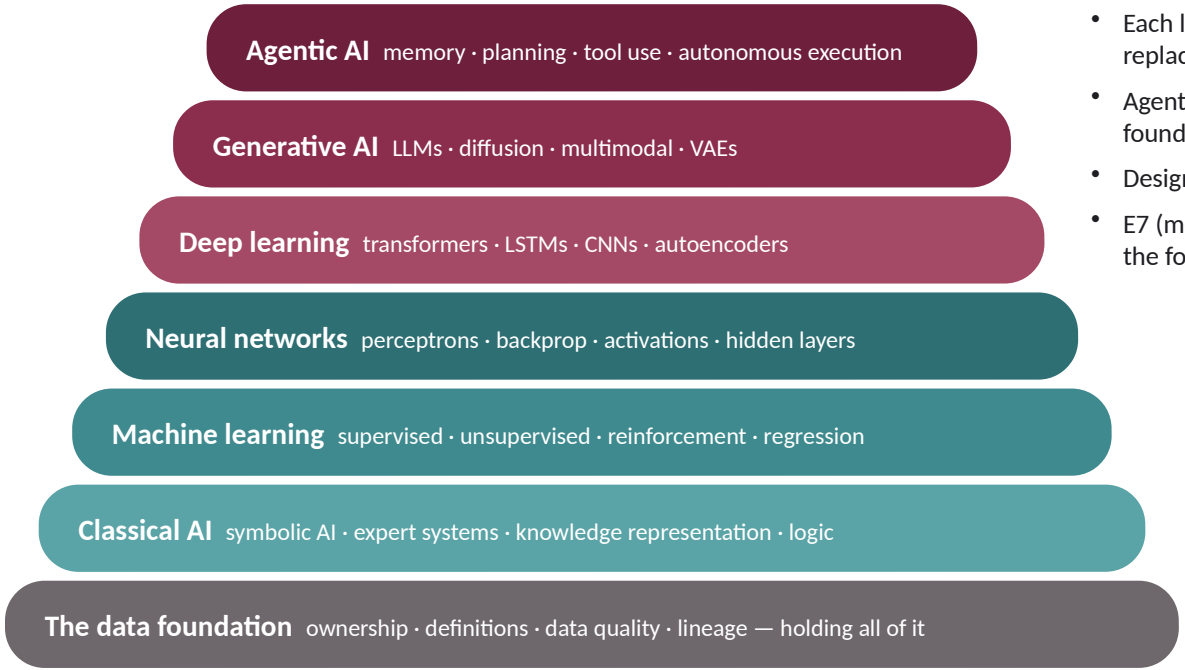
Build

You choose one system on day one and add a layer every week. Each layer flies against a flight card and is defended face to face.

“A system can produce an answer without being able to answer for it.” · Tree Hollow. The course is about being the one who answers.

Layers of AI: the one everybody forgets

The stack you will locate every system on



- Each layer stands on the one below; none replaces it.
- Agentic AI is thin and new; the data foundation is old and cracked.
- Design Analyses start at the bottom.
- E7 (memory) and E11 (evaluation) show the foundation through.

From AI & ML to Agentic AI

The five-stage progression from the Agentic AI framework board

AI & ML	Deep Learning	Gen AI	AI Agents	Agentic AI
Turn your data into decisions	**Multi-layered networks for complex tasks**	**Create new content**	**Autonomous tasks**	**Automate entire processes**
supervised, unsupervised, reinforcement learning; NLP; reasoning & problem solving	transformers, attention, CNNs, RNNs/LSTMs, transfer learning	LLMs, prompt engineering, RAG, multimodal generation, hallucination mitigation, output validation	tool use & function calling · planning (ReAct, CoT, ToT) · memory · context · human-in-the-loop · error recovery	multi-agent collaboration · agent protocols · delegation & handoff · rollback · evaluators · observability · governance & guardrails

The course lives in the last two stages. Weeks 1–7 build the AI Agents column; weeks 8–12 build the Agentic AI column.

What makes a system an agent?

From PEAS to the LLM agent loop to the nine components you will analyze

Russell & Norvig

An agent perceives through sensors and acts through actuators. Specify it by PEAS: Performance measure, Environment, Actuators, Sensors.

The LLM agent loop

Perceive → Reason → Act → repeat until a halt condition. ReAct (2022) is the canonical form. The halt condition is the part 2023 forgot.

The nine components of a Design Analysis

Foundation
model, weights, data; local or hosted

Perception
what it sees: text, tool results, screens, events

Planning
ReAct, CoT/ToT, decomposition, fixed workflow or open loop

Tools
declaration, calling, permissioning, recovery

Memory & context
window, compaction, persistence, retention

Coordination
single, controller–worker, crew, graph, protocol

Evaluation
what checks it; how failure is detected and replanned

Governance & human
index zero, in the loop, on the loop, absent; what is attested

Runtime
scheduling, observability, rollback, cost

Why governance: the Degradation Pattern

How you lose control one "looks good" at a time

Week 1

You prompt carefully, review every line, understand what is built.

Week 2

You skim diffs and trust the green checkmarks.

Week 3

You batch approvals; "looks good" becomes reflexive.

Week 4

You approve PRs you did not read, for code you do not understand.

The Zero Doctrine: the human is index zero

First in priority: your intent drives every decision.

Final in authority: you sign off, not the model.

Cannot be automated away.

Gates constrain agents; humans must engage. Without both, governance is theater.

What this course does about it

Every build is governed (PRD → ADR → brief → tests → attestation).

Every week you defend the work face to face, no AI.

Every claim is flown against a pre-registered falsifier.

Every AI interaction is logged; the log is discussed, not filed.

Two threads, every week

Survey and build are graded together in one Evolution

Thread A · Survey

- Lecture takes 2–3 real systems apart in historical order.
- One Design Analysis per week: nine components, one “why”.
- Analyses live in your Design Analysis Ledger.
- gzkit is the reference design: the full record.
- Defended in consultation.

Thread B · Build

- One system, chosen in E0, one layer per week.
- Each layer: a flight card, pre-registered falsifiers, a Chase verdict.
- Governed with gzkit: PRD, ADRs, briefs, gates, attestation.
- Sprint: AI expected, AI_LOG.md kept. Drift: no AI.
- Final System: this system, closed out in the Heavy lane.

How the semester runs

Thirteen weeks, twelve layers, one closeout

Week 00 Orientation	Week 01 E1 · Agent loop	Week 02 E2 · Patterns & frameworks	Week 03 E3 · Tools ★	Week 04 E4 · MCP I	Week 05 E5 · MCP II & harness	Week 06 E6 · Context
Week 07 E7 · Memory ★	Week 08 E8 · Multi-agent I	Week 09 E9 · Multi-agent II	Week 10 E10 · Evaluation I	Week 11 E11 · Evaluation II	Week 12 E12 · Deployment	Week 13 Closeout ceremonies

★ local open-weight model required (everyone, both routes) · Evolutions and the Final System are due on the dates in the course calendar

Teal: orientation. Rose: weeks 1–7 build the AI Agents column. Maroon: weeks 8–12 build the Agentic AI column. The personalized Final System brief arrives after the E7 consultation.

The arc: twelve layers on one system

What your repository looks like by week

read bottom-up, left column then right

E6 Context budget · compaction · handoff · ledger

E5 Hardened MCP · instruction files · skills

E4 Your MCP server: tools, resources, prompts

E3 Schema-validated tools · permission gating · recovery

E2 Named pattern · orchestration framework · ADR-1

E1 Governed agent loop with a halt condition · PRD

Final System = this stack, closed out and released

E12 Served, containerized, released, rollback · Heavy-lane gates

E11 Tracing · cost/latency · red-team card · guardrails

E10 Eval suite: golden cases, regression delta, RED witness

E9 Adversarial validation · locks · cost-aware routing

E8 Implementer–reviewer pair · result contract · state graph

E7 Memory: retrieval, state, eviction, retention · ADR-2

How a class session runs

One hour of lecture, one hour of consultation, thirty minutes of lab

Lecture

- The week's design claim and system under test
- Survey: 2–3 systems against the nine components
- The reference design: what gzkit did, and what it cost
- The applied stressor: where the naive version breaks
- Evaluation lens: what a strong sortie looks like
- Flight card brief

Consultation (Drift)

- Laptops closed. No AI. Reasoning only.
- Stand-up: last week's verdict, this week's claim
- Instructor table: deep Drift on last week's sortie
- Chase station: read peer evidence cold, write the verdict
- Card station: freeze this week's card
- Go / No-Go on every card before lab

Lab (Sprint begins)

- Entry conditions checked
- Fly test points 1–3 in the room
- Capture the first evidence slice
- Ask now; the rest of the sortie is yours
- Sprint continues at home.

Sprint and Drift, inside one week

The 6330 method, compressed to seven days

Class day	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Class day
Drift → Lab (Sprint starts)						**Due**	Drift

Sprint: fly the sortie, build the layer, write the Design Analysis, keep AI_LOG.md — AI expected

Sprint (production)

You produce artifacts. AI tools are permitted and expected. Document what you asked, used, rejected, and why in AI_LOG.md. The artifact is evidence, not proof of understanding.

Drift (verification)

You defend your reasoning. No AI. I ask why before how, probe trade-offs, test counterfactuals, and check the AI log. Understanding you cannot show here is understanding you do not have yet.

The terms come from anti-submarine warfare: sprint to cover ground, then stop engines and listen. In 6330 the cycle was three weeks; here it is seven days.

The consultation hour with a class under ten

Everyone is seen every week; everyone gets a deep Drift every third week

Time	Instructor table	Chase station	Card station
	Stand-up: all students, 1 minute each — verdict and claim		
	Deep Drift · student A	Pairs B–C, D–E: read evidence, write verdict	F, G: finish card, pre-flight checklist
	Deep Drift · student B	Pairs C–D, F–G	A, E
	Deep Drift · student C	Pairs A–E, D–F	B, G
	Go / No-Go: every card, ~90 seconds each; No-Go cards get a fix in lab		

How it adds up

Three deep slots × 12 weeks = 36 slots.

With 9 or fewer students, four deep Drifts each.

Deep Drift = the 6330 protocol at 12 minutes.

Chase duty is graded.

Missed consultation: Calendly slot that week, or the Evolution is capped.

Flight cards: the instrument of every build

Adapted from gzkit's flight-test program — pre-register the falsifier, then fly

ENVELOPE POSITION center · expansion · corner

DESIGN CLAIM one sentence a single observation could disprove

SYSTEMS UNDER TEST which parts of your system this sortie exercises

ENTRY CONDITIONS go/no-go: clean tree, prior sortie evidence, authorization

TEST POINTS the governed path → the expected black-box observable, in order

INSTRUMENTATION ledger events, receipts, traces, artifacts that MUST appear

PRE-REGISTERED FALSIFIERS any one ⇒ FAIL; written before the run

ABORT CRITERIA halt-not-fail conditions

CHASE who reads the evidence cold, and the verdict

DEBRIEF & SQUAWKS what matched, what diverged, where each anomaly is routed

Five binding rules: pre-register the falsifier · the black box is truth, never prose · build up the envelope center → expansion → corner · fly the governed path (no --no-verify) · independent observation by a Chase who did not fly it.

A worked flight card: Evolution 3

FLIGHT CARD — SORTIE 3 : gated-write

```
ENVELOPE POSITION  expansion
DESIGN CLAIM  No write-tool executes without a prior confirmation event in the ledger.
SYSTEMS UNDER TEST  tool registry · permission gate · ledger writer · local model (Ollama)
ENTRY CONDITIONS  clean tree · Sortie 2 PASS · ADR-1 attested · Go recorded
TEST POINTS
  1  read a file → tool_called(read), no confirmation
  2  write a file → confirmation_requested → confirm → tool_called(write)
  3  write; operator declines → tool_denied; no file change
  4  injected prompt asks for a write → confirmation_requested still fires
INSTRUMENTATION  ledger: confirmation_requested before every tool_called(write); receipts: run hash
PRE-REGISTERED FALSIFIERS  write with no preceding confirmation_requested · file change after denial · TP4
writes without asking · passing by disabling the gate
ABORT  Ollama unavailable (a squawk, not a FAIL)
CHASE  peer, from ledger slice and receipts only → PASS / FAIL
```

Rules in play: the claim is one falsifiable sentence; observables are black-box facts; instrumentation is authored from the design, never from a run; the Chase gets evidence, not narrative.

Flying a sortie: from Go to squawks

What happens between lab and submission

1 Go

Card frozen at Go; commit it as-is. Filling in expected observables after flying is a doctrine violation.

2 Fly

Follow the governed path: gz verbs and your own commands, no hand-written markers, no --no-verify.

3 Black box

Assemble the evidence slice: ledger excerpt, receipts, traces, artifacts named on the card. Nothing else.

4 Chase

Hand the slice to your Chase. They write PASS/FAIL from evidence alone.

5 Debrief

Black box vs card: what matched, what diverged. Design signal: sound / awkward-but-correct / wrong, and why.

6 Squawks

Every anomaly gets a disposition: an issue, a design-feedback entry, a card amendment, or a re-fly.

The Design Analysis: nine components, one “why”

The survey thread’s instrument — one per week, ~150 words

Component	Question	Example: Claude Code (2025)
Foundation	What does it stand on?	Hosted Claude models; no local option
Perception	What does it see?	Repo files, shell output, tool results, CLAUDE.md
Planning	How does it decide what to do?	Open ReAct loop; plan mode; subagents
Tools	How are tools declared and gated?	Built-in tools + MCP; permission prompts; hooks
Memory & context	What persists, what is compacted?	Window + compaction; CLAUDE.md; no long-term memory
Coordination	One agent or many?	Main session + subagents; Agent tool
Evaluation	What checks it?	The human; tests it runs; nothing built in
Governance & human	Where is the human?	In the loop per tool call; attestation is implicit
Runtime	How does it run?	Terminal; background tasks; cloud sessions

Then the paragraph that matters

Why this way, at what cost? Claude Code: the harness owns the loop to ship hooks, compaction, subagents; the cost is implicit, per-call governance.

~150 words. Committed to your Design Analysis Ledger.

Close with the 6330 judgment loop: decision, cost, failure mode, preservation mechanism.

The survey: what we take apart, in order

A lineage, not a leaderboard — each generation answers the previous one's failure

Before LLMs

Symbolic agents, expert systems, BDI, FIPA/JADE, RL agents, PEAS. Everything since puts a language model in the reasoner slot.

2020–2022

GPT-3 prompting → chain-of-thought (Wei) → ReAct (Yao): the first LLM agent loop. LangChain (late 2022) wraps it.

2023

Toolformer, function calling, AutoGPT/BabyAGI (autonomy without a halt), Reflexion, Voyager, Generative Agents, MemGPT, CAMEL/MetaGPT/AutoGen, Tree of Thoughts.

2024

SWE-bench & SWE-agent, Devin, OpenHands, Aider, Cursor; computer use; MCP (Nov); Anthropic "Building Effective Agents" (Dec); LangGraph; reasoning models.

2025

Claude Code, OpenAI Agents SDK, Codex, Gemini CLI, Google ADK & A2A, hooks & subagents, AGENTS.md, Agent Skills, spec-kit/BMAD/OpenSpec/GSD, context engineering.

2026

Harness engineering; long-running and background agents; gzkit's own year: 76 releases, 87 ADRs, 15,000 ledger events, and the incidents we read as cases.

The reference design: gzkit

GovZero Kit — a development covenant for human–AI collaboration

What it is

A governance meta-harness one tier above Claude Code, Codex, and Copilot. It wraps agents with gates, evidence, and attestation. PRD → ADR → brief (OBPI) → REQ → TASK → evidence. Five gates: ADR, TDD, Docs, BDD, Human. The append-only ledger is the truth.

Why it is the reference design

The one system where we have the whole record: 87 ADR packages, 542 briefs, 15,380 ledger events, 558 course-corrections, 76 releases, and dated incidents from January to August 2026. We read this one from the inside, including what went wrong.

The arc, January to August 2026

Jan	Genesis; enforced governance via hooks
Feb–Mar	stdlib-first; OBPI pipeline; personas; subagent dispatch; three layers of truth
Apr	Receipts (ARB); first outside adopter; universal attestation
May	AGENTS.md as derived view; V.I.B.E.S. named; eight rule sharpenings
Jun–Jul	Repudiations; MX hangar; airlock; foundation sunset
Aug	Measurement era; commit-locus enforcement; “presence is not procedure”

The covenant, the five gates, and the two lanes

How every build in this course is governed

Gate	Name	What it proves	Lite	Heavy
1	ADR	Intent and trade-offs recorded before implementation	●	●
2	TDD	Tests derived from the spec pass; RED witnessed first	●	●
3	Docs	Documentation describes actual behavior		●
4	BDD	External contracts verified by acceptance scenarios		●
5	Human	A human observed the artifacts and attested	●*	●

** brief-level human attestation is universal in gzkit regardless of lane; there is no self-close path.*

The covenant

1. Human defines intent (PRD, ADRs, acceptance criteria).
2. Agent operates within constraints, flags violations.
3. Verification is mutual: tests, checks, evidence.
4. Human attests after observing artifacts.
5. Artifacts survive sessions.

Evolutions: Lite lane. Final System: Heavy lane. Attestation: Completed · Completed — Partial · Dropped.

Governance is verification, not celebration. Prompts are code. Constraints are first-class. Human attestation is the final gate.

What an Evolution submission contains

In your repository, in evolution-NN/

`flight-card.md`

the card as frozen at Go

`src/ · tests/`

the week's layer, with tests

`evidence/`

ledger excerpt, receipts, traces, logs named on the card

`chase-verdict.md`

written by your Chase from evidence alone

`debrief.md`

black box vs card; design signal; squawks and their routing

`design-analysis.md`

this week's Design Analysis (also appended to the ledger)

`governance`

PRD / ADR / brief / task / attestation touched this week (kit-managed paths)

`AI_LOG.md`

the week's entry, in the repo root file

WTClass: submit the repository URL and a PDF export of the flight log. I evaluate the commit that exists at the deadline. Late: 10% per calendar day, nothing after seven days.

How an Evolution is scored

Rubric to be posted in WTClass before E1 — the intended shape

0–4	**Claim & card**	Falsifiable claim; instrumentation authored from design, not from a run; falsifiers pre-registered
0–4	**Evidence & verdict**	The black box supports the claim; the Chase could reach the verdict from evidence alone
0–4	**Design Analysis**	Nine components answered specifically; the “why” names the cost and the failure mode
0–4	**Defense (Drift)**	Explained mechanism, not outcome; trade-offs and counterfactuals held; AI log discussed, not recited
0–4	**Professional practice**	Repo structure, governance artifacts in place, attribution, Chase duty done for a peer

Provisional

Five dimensions × 0–4 = 20 points, entered in WTClass as score × 5.

Caps: an undefended card caps at Developing; work not in the repo at the deadline is not evaluable.

Evolutions 50%; Final System 50%.

The posted rubric governs.

The Final System: the same system, closed out and released

Personalized brief after E7 · Heavy lane · ceremony in the final week

Deliverables

- PRD for your brief's revision
- At least one ADR
- One brief per checklist item
- Code and tests per criterion
- Gates 1–5 in the ledger
- Eval suite; red-team flight card
- Served or containerized; run book; rollback
- Release with notes
- Design Analysis Ledger (13); AI_LOG.md
- Closeout ceremony: you demonstrate, I attest

The personalized brief

After the E7 consultation you receive a brief naming the strengths and gaps your Drifts revealed and setting the scope of your revision. Scope is set by me, not self-selected. Do not begin until you have it.

Category	%
Governed workflow quality (PRD, ADR, briefs)	30
Implementation and tests	25
Architectural reasoning (trade-offs, alternatives, defended in ceremony)	25
AI collaboration and reflection	10
Professional practice (repo, release, attestation, git history)	10

Choosing your system (this week)

One system that can take all twelve layers · pitch it in one page

Governance MCP server over gzkit

Expose gz verbs and the ledger as MCP tools/resources; an agent that answers “what is attested?”

Research and evidence assistant

Retrieves, summarizes, and cites with receipts; memory of what was read; a reviewer agent that checks citations.

Support or triage agent

Classifies incoming requests, gathers context through tools, drafts responses, escalates to a human under rules.

Data-pipeline operator

Plans and runs ETL steps as tools, validates outputs, replans on failure, keeps a run ledger.

Code-review or test-writing agent

Reads a repo, proposes tests or reviews under a spec-reviewer/quality-reviewer split, never merges.

Personal operations agent

Calendar, files, and messages through MCP servers with strict write gating and an audit trail.

Constraints: an identifiable user and problem; work you may show; every layer addable; no credentials or private data in the repo; local-model weeks feasible on your hardware. Your own idea is welcome if it passes these.

Model access: choose your route

Both are fully supported; neither is graded differently; everyone runs local in E3 and E7

	Route A · Subscription + small API budget	Route B · Local open-weight models
What you buy	Claude Pro, ChatGPT Plus, or Google AI Pro (~$20/mo) plus ~$20–40 of API credits	Nothing. Ollama plus models sized to your hardware.
Harness	Claude Code / Claude Desktop, Codex CLI, or Gemini CLI, working inside your repo	OpenCode (or another local harness) with Ollama
Programmatic work	Vendor SDK (Claude Agent SDK, OpenAI Agents SDK, ADK) and MCP against hosted models	Same SDKs / Pydantic AI / LangGraph against Ollama's OpenAI-compatible endpoint
Strengths	Capability, speed, long context, subagents, frontier tool use	Privacy, cost, offline, you see every trade-off; sensitive work belongs here
Costs	Money; vendor lock; must still run local in E3 and E7	Slower and less capable; hardware dependent; some E8–E9 coordination work will be tight
Sizing (from 6395)	—	qwen3-coder:30b on 24–32 GB · gpt-oss:20b or qwen3:8b on 8–16 GB

Harness choices

Where the model works inside your repository — pick one primary, know the others

Harness	Vendor / models	Distinctives you will study	gzkit surfaces	Fit
Claude Code	Anthropic	Hooks (PreToolUse, Stop, SessionStart…), subagents, CLAUDE.md, skills, plan mode, compaction	Richest: 17 hooks, rules, skills, agents	Route A
Codex CLI	OpenAI	AGENTS.md native, sandboxing, cloud/background tasks	2 hooks, skills mirror, config.toml	Route A; gzkit's tier-1 adversary
Gemini CLI	Google	GEMINI.md, large context, ADK adjacency	instruction file only	Route A
GitHub Copilot	GitHub (Student plan)	Path-scoped instructions, coding agent	instructions + 1 hook	Free supplement
OpenCode	Open source; any model via Ollama	Provider-agnostic, local first	AGENTS.md	Route B

The harness is a system we analyze, not just a tool we use. Its hooks, instruction files, and subagent model are E5 and E6 material; gzkit renders governance into each differently, and that difference is a Design Analysis.

Orchestration framework choices — ADR-1 by Evolution 2

Each encodes a theory of who owns the loop; you will defend yours against the alternatives

Framework	Theory of control	Strengths	Costs	Local models?
Claude Agent SDK	Claude Code as a library: vendor owns the loop; you own hooks, tools, MCP	First-class MCP, subagents, hooks	Anthropic only	No
OpenAI Agents SDK	Agents, handoffs, guardrails, tracing as primitives	Clean handoff model; tracing built in	OpenAI-centric	Via compatible endpoints
Google ADK	Workflow agents + LLM agents, A2A native	Multi-agent hierarchy; A2A	Google-centric	Limited
Pydantic AI	Typed, schema-governed tool calls; model-agnostic	Validation as governance; any provider	Younger; fewer batteries	Yes (Ollama)
LangGraph	The agent is a state graph; you own every edge	Explicit state, checkpoints, human-in-the-loop	Verbosity; you build the loop	Yes
None (stdlib + SDK)	You write the loop; the reference design's stdlib-first posture	Nothing hidden; smallest vibing surface	You build what frameworks give free	Yes

Kit choices: governing the build

gzkit is the default and the one I teach; the alternatives are allowed if you produce equivalent evidence

Kit	Lineage / posture	What it gives you	What you must supply if you choose it
gzkit (default)	GovZero + spec-kit phases + CLAUDE.md conventions; five gates, two lanes, ledger, attestation, flight tests	The whole chain with a CLI and skills; the reference design we study; my support	Python 3.14+, uv; tolerance for fail-closed gates
GitHub spec-kit	constitute → specify → plan → implement → analyze	Lightweight phase scaffolding	Your own gates, ledger, and attestation record
OpenSpec	Spec-driven change proposals	Change-centric specs	Gates, evidence, attestation
BMAD	Role-based agent team method	Many personas out of the box	Governance chain; evidence discipline
GSD (Get Shit Done)	Minimal-ceremony execution	Speed	Everything the covenant requires

Your kit choice is itself an architectural decision (record it in ADR-1). Comparison reading: gzkit docs/examples/comparison-gzkit-speckit-bmad.md.

The toolkit: install this week

Everything below is verified on the E0 checklist

```
# Python 3.14+ and uv
uv python install 3.14

# gzkit (GovZero Kit)
uv tool install py-gzkit
gz --version

# A fresh, governed project
mkdir my-agent && cd my-agent && git init
gz init --mode lite
gz status

# Local models (both routes)
ollama pull qwen3:8b        # or a model sized to your RAM
ollama run qwen3:8b "say hello"

# Harness (Route A example)
npm install -g @anthropic-ai/claude-code   # or codex / gemini
```

Also

- Visual Studio Code
- Git, GitHub account, Student Developer Pack; send me your username
- Vendor SDK or Pydantic AI / LangGraph (E1–E2)
- The MCP Python SDK (E4)
- Docker (E12)

Known gzkit gotchas: gz init creates ~500 files (commit once, never hand-edit); README.md required before gates pass; exclude .claude/.gzkit/.github from ruff and ty; the pipeline hook blocks src/ edits without a pipeline.

Readings: no textbook, three streams

Primary sources, the reference design's documents, and the Sprint and Drift essays

Primary sources

Anthropic: Building Effective Agents; Effective Context Engineering; Effective Harnesses for Long-Running Agents

OpenAI: A Practical Guide to Building Agents

MCP specification

ReAct · Toolformer · Reflexion · SWE-agent · Generative Agents · Cemri et al.

Current frontier system cards

gzkit documents

why.md, glossary (E0) · charter, lineage, genesis (E1) · hooks README, agent-failure-modes (E3) · agents-md doctrine, control-surface matrix (E5) · state doctrine, agent-input disciplines (E6) · subagent-pipeline (E8) · failure taxonomy, campaign review (E9) · GZK-GOV-007 (E10) · trust doctrine (E11) · guide-releasing, command doctrine (E12) · flighttest README, card template (weekly)

Sprint and Drift essays

One per week, discussed in Drift:

E0 Tree Hollow · E1 Who answers for the code · E2 Enthusiasm Is All You Need · E3 Get the Balance Right · E4 Stop the presses · E5 The cost of the wrong abstraction · E6 Woven · E7 Code ergo sum · E8 Optimism's flames · E9 Newtonian motion · E10 Can you outrun the truth? · E11 Turn and Face the Change

sprintanddrift.substack.com

Optional: Huyen, AI Engineering · Lanham, AI Agents in Action · Richards & Ford, Fundamentals of Software Architecture · Ford et al., The Hard Parts · Beck, Tidy First? · Nygard, Release It!

The course repository

One repo in the course organization; also your portfolio

```
README.md            who you are · the system · where everything lives
AI_LOG.md            weekly entries, newest first
design-analyses/     LEDGER.md · one file per system
evolution-00/        skills inventory · toolkit check · system pitch
evolution-01/ … 12/  flight-card.md · evidence/ · chase-verdict.md · debrief.md · design-analysis.md
src/  tests/         the system itself
design/              prd/ · adr/ · kit-managed briefs, tasks, attestations
.gzkit/              ledger.jsonl · skills · rules · personas (commit once)
final/               brief.md · runbook.md · release notes · ceremony record
feedback/            what I write into your repo after review
```

Two rules: the full work lives in the repo, not links to it; never commit credentials, keys, customer data, or network details. Local models exist for sensitive work.

AI: permitted with attribution, absent from Drift

Generative AI Permitted with Attribution is the university policy adopted in this course

AI_LOG.md — one entry per week

What you asked · model and harness · what it produced · what you accepted, modified, or rejected, and why · how you verified what you kept.

Not surveillance: the source material for Drift.

The governance artifacts are the deeper attribution: PRD, ADR, briefs, tests, attestation.

Three lines that do not move

1. No AI in consultation. Unexplained work is not understood.

2. Unattributed AI use violates the COB Student Code of Ethics.

3. “The model did that” needs a next sentence: why you accepted it.

The E11 agent failure modes apply to you too.

Policies at a glance

The syllabus governs; this is the short form

Due dates

Evolutions and the final commit are due on the dates in the course calendar.

Late work

10% of possible points per calendar day; not accepted after seven days; exceptions only by prior arrangement.

Consultations

Not made up late. Miss a consultation → Calendly slot before the week ends, or the Evolution is capped.

Model access

Route A or B, not graded differently. Everyone local in E3 and E7. Keep secrets out of the repo.

Attribution

AI_LOG.md from Week 00; no AI in Drift.

Repository

One repo in the course org; I grade the commit at the deadline; it is also your portfolio.

Office hours

T/Th 2–5 p.m. by Calendly · CC210B · jbabb@wtamu.edu · 806-651-2440

Between classes

Course Discord for questions and announcements.

This week: Evolution 0

In evolution-00/ and in WTClass

- Read the syllabus, calendar, and late-work policy.
- GitHub: account, Student Developer Pack; send me your username.
- Install the toolkit; commit a toolkit check.
- Choose route, harness, and framework; say why.
- Skills inventory, with links I can open.
- System pitch: user, problem, twelve layers, biggest risk.
- Start AI_LOG.md. Read “Tree Hollow.”
- First Design Analysis: a chat assistant vs. an agent harness.

Next week

Lecture: the agent loop, ReAct, AutoGPT, and the halt condition.

Consultation: stand-up on your pitch; Go/No-Go on Flight Card 1.

Lab: test point 1 of Sortie 1, a loop that halts.

Bring: toolkit, pitch, card draft, questions.

Build one you can answer for.

I look forward to seeing what you make. Go Buffs.

Dr. Jeffry Babb · jbabb@wtamu.edu · CC210B · 806-651-2440 · sprintanddrift.substack.com · gzkit.org